

Incremental Development of a Shortest-Path Pick-and-Place Controller for an Eight-Station CNC Tool Magazine on a Micro820 PLC

Prajjwal Dutta

Ira A. Fulton Schools of Engineering
Arizona State University
Tempe, AZ, USA
pdutta9@asu.edu

Abstract—This report documents the design, implementation and verification of an automated pick-and-place controller for an eight-station CNC tool magazine, realised on an Allen-Bradley Micro820 programmable logic controller and programmed in Connected Components Workbench. The application is built in six incremental stages, beginning with a single-rung seal-in latch and ending with an array-driven tool sequencer that selects the shorter of two rotational arcs to each target station, delays at the transfer point, counts processed parts, and completes its current cycle after a stop request. Six user-defined function blocks written in IEC 61131-3 Structured Text encapsulate binary sensor decoding, thumbwheel debouncing, the rotation-direction decision, target detection, array loading and array sequencing; the main program remains a readable sixteen-rung ladder diagram. The rotation decision uses a modulo-based antipode test whose asymmetry on an even-order wheel is characterised and bounded. Three self-contained browser simulators were developed to exercise the algorithm and the ladder logic without occupying the hardware trainer. Verification was performed by reading the Structured Text emitted by the CCW QuickLD compiler for each committed project, which exposed three implementation findings that are reported in full: an unimplemented Lab 6 project, an off-by-one binding of the rotation function-block inputs, and an output-coil write that overwrites a function-block result within the same scan.

Index Terms—programmable logic controller, IEC 61131-3, ladder diagram, structured text, tool magazine, pick and place, shortest path, Micro800, Connected Components Workbench, industrial automation education

I. INTRODUCTION

Sequential process control is the workhorse of discrete manufacturing, and the programmable logic controller (PLC) remains its dominant implementation platform. Teaching it well is difficult for a practical reason: realistic industrial behaviour normally requires industrial-scale equipment. Bench trainers that reproduce the *control* problem without the machine solve this, and the Industrial Concepts CNC Tool Magazine Control Unit is one such device.

The trainer presents an eight-station rotating tool wheel. A magnet fixed to the wheel passes over eight discrete position sensors, a four-bit binary-coded decimal (BCD) thumbwheel switch lets an operator name a station by number, and two digital outputs drive the wheel forward and in reverse. Despite

the small input and output count, the control problem is genuinely non-trivial: the controller must decide *which direction* to rotate so that travel to the next tool is minimised, detect arrival, carry the tool back to a transfer station, and repeat the operation over a sequence the operator enters at run time.

This report describes a complete implementation of that application on an Allen-Bradley Micro820 controller, developed across six laboratory stages following the trainer's reference lab documentation [1]. The work is organised around three claims. First, that an incremental construction, in which each stage is independently testable on hardware, is an effective structure for a non-trivial PLC application. Second, that partitioning between ladder diagram and Structured Text along the boundary of *arithmetic versus interlocking* keeps a sixteen-rung program readable while supporting genuinely algorithmic behaviour. Third, that reading the Structured Text emitted by the vendor's ladder compiler is an economical and effective verification technique; in this work it surfaced three defects that visual inspection of the ladder diagram did not.

Section II describes the plant and controller. Section III develops the software architecture. Section IV analyses the shortest-path algorithm and bounds its known asymmetry. Section V walks through the six implementation stages. Section VI describes the simulation tooling. Section VII reports the verification method and its findings. Section VIII concludes.

II. SYSTEM DESCRIPTION

A. Plant

The tool wheel carries eight stations at 45° intervals, numbered 1 through 8. One station is designated the *base*, or place, position: the point at which a tool is handed to or received from the machine spindle. The remaining stations hold tools available for pickup. A magnet on the rotating platter triggers a sensor as it passes each station, so the controller observes position as an eight-line one-hot code rather than as an encoder count. The wheel is driven by a single motor with forward and reverse contactors; there is no speed control and no absolute position feedback beyond the eight sensors.

TABLE I
CONTROLLER I/O ASSIGNMENT

Designator	Alias	Function
<i>Embedded digital inputs</i>		
_IO_EM_DI_00	Stop	Stop push button
_IO_EM_DI_01	Start	Start push button
_IO_EM_DI_02..05	BCD1,2,4,8	Thumbwheel bits, binary weights
_IO_EM_DI_06..11	Position1..6	Wheel position sensors 1–6
<i>Plug-in digital inputs (2080-IQ4, slot 1)</i>		
_IO_PI_DI_00	Position7	Wheel position sensor 7
_IO_PI_DI_01	Position8	Wheel position sensor 8
<i>Embedded digital outputs</i>		
_IO_EM_DO_00	Light1	Indicator lamp 1
_IO_EM_DO_01	Light2	Indicator lamp 2
_IO_EM_DO_02	FWD_Motor	Forward motor relay
_IO_EM_DO_03	REV_Motor	Reverse motor relay

B. Controller and I/O

The controller is an Allen-Bradley Micro820 2080-LC20-20QBB with twelve embedded 24 V DC digital inputs and seven source outputs. Because the application needs fourteen discrete inputs, a 2080-IQ4 four-point digital input plug-in module occupies slot 1 and supplies position sensors 7 and 8. The controller is programmed over Ethernet from a workstation running Rockwell Connected Components Workbench (CCW).

Table I gives the complete input and output assignment. CCW’s alias mechanism maps the physical designators onto process-meaningful names, and every rung and function block in the application refers to the aliases.

C. Operator interface constraint

The trainer provides only two push buttons and one BCD thumbwheel. There is no keypad and no display, so entering a multi-station tool sequence must be encoded in thumbwheel gestures. The adopted convention, developed in Section V, uses thumbwheel position 9 as a sequence reset and a downward pass through position 0 as a store pulse. This constraint is representative: minimising I/O point count is a routine commercial pressure, and it routinely pushes complexity from the panel into the program.

III. SOFTWARE ARCHITECTURE

A. Language partitioning

CCW supports Ladder Diagram (LD), Structured Text (ST) and Function Block Diagram. The application partitions along a single rule: *ladder for interlocking and sequencing, Structured Text for arithmetic and decision logic*. Ladder is the natural notation for seal-in latches, motor interlocks and one-shot pulse routing, and it is what a maintenance technician expects to find when troubleshooting. Conversely, expressing a modulo operation, a weighted binary-to-decimal conversion or a nested conditional in ladder produces a wide, opaque diagram; those belong in ST.

The result is six user-defined ST function blocks called from a sixteen-rung ladder main program (Prog1). Table II summarises their interfaces.

B. Signal flow

Fig. 1 shows the dataflow. Two decoders convert the physical layer into integers. Those integers, together with the current array element, feed the two-stage direction decision. The direction decision reduces to a single Boolean that the two motor rungs consume through complementary contacts, which guarantees mutual exclusion of the forward and reverse contactors in the program as well as in the panel wiring.

C. Scan-cycle discipline

Two recurring hazards shape the ST code. The first is that a PLC re-executes its program every scan, so any input that remains asserted for longer than one scan, which is every mechanical input, will retrigger an action repeatedly. Each event in this application is therefore converted to a single-scan pulse using the standard delayed-copy idiom:

```
Inc_EN      := BCD0_Load_in AND NOT BCD0_delayed;
BCD0_delayed := BCD0_Load_in;
```

The second is contact bounce on the thumbwheel, which is a rotary make-before-break switch and produces transient invalid codes while turning. FB_BCD_to_Dec handles this with a self-resetting 100 ms on-delay timer that emits one sampling pulse per period, then requires two consecutive equal samples before asserting Debounced_EN.

```
IF BCD_en THEN
  TimeDelay := T#100ms;
  TON_1((NOT Debounce_BCD_sw), TimeDelay);
  Debounce_BCD_sw := TON_1.Q;
  IF Debounce_BCD_sw THEN
    BCD_convert := ANY_TO_UINT(BCD1_in)*1;
    BCD_convert := BCD_convert + ANY_TO_UINT(BCD2_in)*2;
    BCD_convert := BCD_convert + ANY_TO_UINT(BCD4_in)*4;
    BCD_convert := BCD_convert + ANY_TO_UINT(BCD8_in)*8;
    PositionSelect_out := BCD_convert;
    IF (PositionSelect_out = PositionSelectT2) THEN
      Debounced_EN := TRUE;
    ELSE
      Debounced_EN := FALSE;
    END_IF;
    BCD_load_out := NOT BCD1_in AND NOT BCD2_in
                  AND NOT BCD4_in AND NOT BCD8_in;
  END_IF;
  PositionSelectT2 := PositionSelect_out;
END_IF;
```

Note the dual use of thumbwheel position 0: it is not a valid station, so it is appropriated as the sequence *store* command, which is how a four-bit switch serves both as a data entry field and as a command key.

IV. SHORTEST-PATH ROTATION

A. Problem statement

Let the wheel stations be $S = \{1, \dots, 8\}$ arranged cyclically. Given a base station $B \in S$ and a target station $T \in S, T \neq B$, the controller must choose a rotation direction $d \in \{\text{fwd}, \text{rev}\}$ minimising the number of station crossings between B and T . The forward and reverse distances satisfy $\delta_{\text{fwd}} + \delta_{\text{rev}} = 8$, so the decision is simply whether T lies in the forward half-arc from B .

TABLE II
USER-DEFINED STRUCTURED TEXT FUNCTION BLOCKS

Block	Key inputs	Key outputs	Responsibility
FB_Position_to_dec	Pos1_in to Pos8_in (BOOL)	PositionValue_out (UINT)	One-hot sensor bus → station number 1..8
FB_BCD_to_Dec	BCD1_in to BCD8_in (BOOL)	PositionSelect_out (UINT), BCD_load_out, Debounced_EN	Debounce thumbwheel; decode BCD; detect position 0
FB_DeterminRotation	PositionSelect_in (base), PositionSequence_in (target), PositionDetectNumber_in	DirectionState (UINT: 1 fwd, 2 rev)	Shortest-arc direction, latched between base crossings
FB_detectTool	as above, plus RotationState_in	DirectionState (BOOL)	Reverse direction at pickup and again at base
FB_LoadArray	BCD0_Load_in, PositionHold_in	PositionSequence_out[1..10], PositionMax_out	Edge-triggered store of a station into the sequence array
FB_SequenceArray	SequenceMax_in, PositionSequence_in, PositionDetect_in	Sequence_Index_out (DINT), PickUp_EN, Place_EN	Advance the array index modulo the stored count; emit event pulses

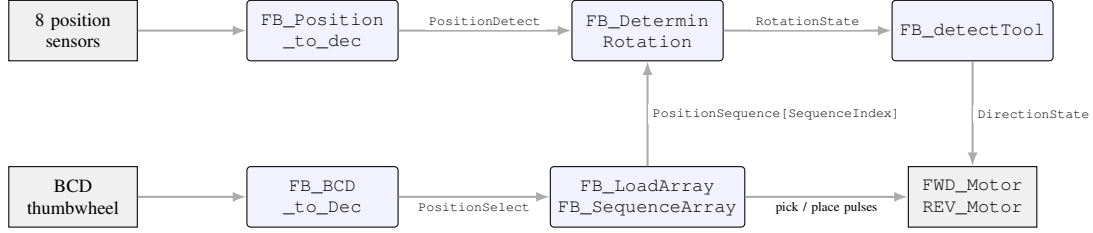


Fig. 1. Dataflow of the tool magazine controller. The two decoder blocks lift the physical layer into integer station numbers; the two direction blocks reduce those integers to a single Boolean that drives the complementary motor rungs.

B. Antipode test

The implementation computes the station diametrically opposite the base,

$$O = ((B + 4) \bmod 8) + 1, \quad (1)$$

where the +1 returns the result to the 1-based station numbering. O partitions the wheel into a forward half and a reverse half. When $O > B$ the forward half is the ordinary open interval (B, O) and the test is a direct comparison. When the wheel wraps past station 8 the station numbers are no longer monotonic across the forward arc, and the implementation instead *excludes* the reverse arc:

```

TempDint := ANY_TO_DINT(PositionSelect_in);
TempDint := MOD(TempDint+4, 8) + 1;
PositionOpposite := ANY_TO_UINT(TempDint);

IF (PositionDetectNumber_in = PositionSelect_in) THEN
  IF (PositionOpposite > PositionSelect_in) THEN
    IF (PositionSelect_in < PositionSequence_in)
      AND (PositionSequence_in < PositionOpposite) THEN
      DirectionState := 1; // forward
    ELSE
      DirectionState := 2; // reverse
    END_IF;
  ELSE
    IF NOT ((PositionSelect_in > PositionSequence_in)
      AND (PositionSequence_in >= PositionOpposite)) THEN
      DirectionState := 1;
    ELSE
      DirectionState := 2;
    END_IF;
  END_IF;
ELSE
  DirectionState := DirectionState; // hold
END_IF;

```

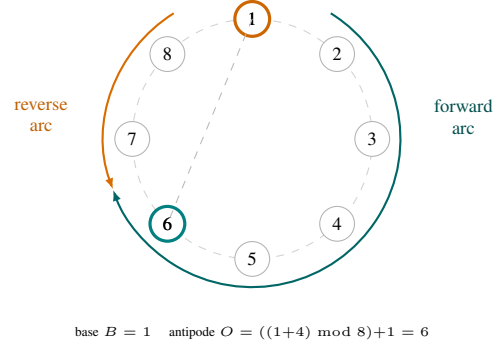


Fig. 2. Antipode partition for base $B = 1$. Stations strictly between B and the antipode O are reached by forward rotation; the remainder by reverse rotation. Because $|S|$ is even, one station lies exactly on the boundary and is assigned by convention rather than by distance.

C. Even-order asymmetry

The block's source comment states the limitation, and it is worth making precise. For an n -station wheel with n even, the antipode O of any base B is itself a station. Removing B and O from S leaves $n - 2 = 6$ stations, which do split evenly into two arcs of three. The station O itself, however, is equidistant: $\delta_{\text{fwd}} = \delta_{\text{rev}} = 4$. The implementation assigns it to the reverse arc through the $\geq$ in the second branch. This is not a defect, since no assignment of a tie can be shorter than another, but it means that for exactly one target per base the choice is arbitrary. A demonstration that appears to take “the long way” at $T = O$ is behaving as specified. Had n been odd, every target would have a strictly shorter arc and the test would be exact.

TABLE III
IMPLEMENTATION STAGES

Stage	Rungs	Capability added
1	1	Seal-in latch; start/stop memory
2	3	Sensor-driven reciprocation between two fixed stations
3	5	Decoded sensors and thumbwheel; operator-selectable target
4	5	Shortest-path direction decision
5	8	Ten-element sequence array with run-time entry
6	16	Transfer delay, parts counter, cycle-stop

The residual observation is therefore bounded: at most 1 of the 7 possible targets per base is direction-ambiguous, and none is chosen incorrectly among strictly-shorter alternatives.

D. Direction latching and reversal

FB_DeterminRotation evaluates only when the wheel sits over the base, guarded by an equality test between the detected station and the base; otherwise it holds its previous value. This matters because the wheel is in continuous motion and the decision inputs are only meaningful at the reference point.

FB_detectTool converts the UINT rotation code into the Boolean the motor rungs consume, and performs the actual reversal: on reaching the target it flips direction so the tool is carried back, and on reaching the base it flips again for the next outbound leg.

```

IF (PositionSequence_in = PositionDetectNumber_in) THEN
  IF (RotationState_in = 1) THEN
    DirectionState := ANY_TO_BOOL(0);
  ELSE
    DirectionState := ANY_TO_BOOL(1);
  END_IF;
ELSIF (PositionSelect_in = PositionDetectNumber_in) THEN
  IF (RotationState_in = 1) THEN
    DirectionState := ANY_TO_BOOL(1);
  ELSE
    DirectionState := ANY_TO_BOOL(0);
  END_IF;
ELSE
  DirectionState := DirectionState;
END_IF;

```

Together the two blocks form a pair of interacting two-state machines: a *rotation* state updated at the base, and a *direction* state toggled at both endpoints of each leg.

V. INCREMENTAL IMPLEMENTATION

Each of the six stages produces an independently downloadable and testable program. Table III summarises the progression. The ladder diagrams reproduced in Figs. 3–10 are not redrawings: they are CCW Document Generator output taken from the built controller projects, so they show exactly what was downloaded and tested on the trainer.

A. Stage 1: Seal-in latch

The seal-in, or latching, circuit is the foundation of nearly all PLC start/stop control: a normally-closed stop contact in series with the parallel combination of a momentary start contact

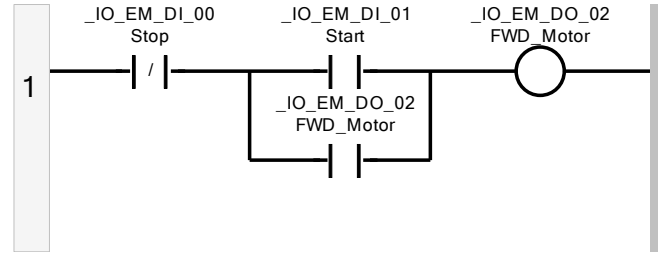


Fig. 3. Stage 1, rung 1 as built. The normally-closed Stop contact is in series with the parallel pair of Start and the output's own feedback contact, which is the seal that provides the memory.

and a feedback contact from the output itself. Reading the compiled ST makes the memory explicit:

```

RunState := (NOT Stop AND Start) OR (NOT Stop AND
RunState);

```

The second disjunct is the seal: once true, the output holds itself true until the stop contact opens the series path. Fig. 3 shows the rung as built. In the committed project the latched output is FWD_Motor rather than an indicator lamp, which is the reference manual's Exercise 2 variant.

B. Stage 2: Reciprocation

Reciprocation replaces the operator's fingers with sensors. A second latch is built with Position1 in the start role and Position2 in the stop role, and the whole rung is gated by RunState:

```

FWD_Motor := ((RunState AND Position1)
OR (RunState AND FWD_Motor)) AND NOT Position2;
REV_Motor := RunState AND NOT FWD_Motor;

```

The complementary form of the reverse rung is the important detail: forward and reverse can never be commanded simultaneously, because the reverse coil is driven from the inverse of the forward coil rather than from an independent condition. This is program-level interlocking, complementing the electrical interlock in the contactor wiring. Fig. 4 shows all three rungs; the structural similarity between rung 1 and rung 2 makes the point that reciprocation is the same circuit with sensors substituted for buttons.

C. Stage 3: Decoding and selection

Stage 3 introduces the two decoder blocks and replaces the hard-wired second sensor with a comparison. FB_Position_to_dec sums the eight one-hot sensor bits with their station weights; the comment in the source correctly notes that although the expression sums all eight terms, only one is non-zero at a time, and the summation form is simply more compact than an eight-way conditional. An equality comparison between the decoded thumbwheel value and the decoded sensor value produces ToolDetected, which takes over the stopping role in the reciprocation rung. The result is that the operator can now select the pickup station at run time without recompiling.

Fig. 5 shows the decoder rung, and is worth studying for what it says about the language partitioning: fourteen physical

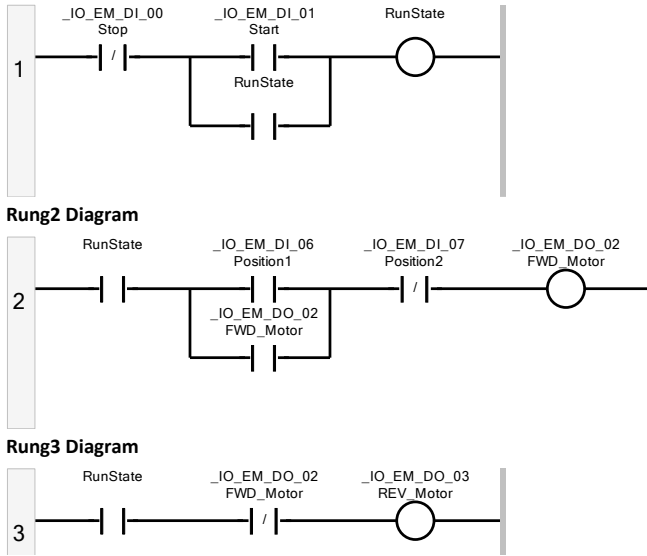


Fig. 4. Stage 2, rungs 1–3. Rung 1 latches RunState from the push buttons; rung 2 is the same seal-in built from Position1 and Position2; rung 3 derives reverse from the inverse of forward.

input points are consumed by two blocks and leave as two integers, and the whole conversion occupies a single rung. Fig. 6 shows the comparison and the motor rungs that consume it.

D. Stage 4: Direction control

Stage 4 introduces `FB_DeterminRotation` and `FB_detectTool`, analysed in Section IV. With direction now computed, the reciprocating latch is no longer needed and the motor rungs collapse to two symmetric rungs:

```
FWD_Motor := RunState AND DirectionState;
REV_Motor := RunState AND NOT DirectionState;
```

This is a significant simplification: sequencing logic that had been distributed across ladder branches now lives in one place, and the ladder retains only the interlock. Fig. 7 shows the direction rung and Fig. 8 the two motor rungs it feeds. Comparing Fig. 8 against Fig. 6 is the clearest single illustration of what moving logic into function blocks buys: the branch structure has disappeared entirely.

E. Stage 5: Sequence array

Stage 5 makes the application useful for a repeating manufacturing process by storing a list of pickup stations. The array `PositionSequence[1..10]` is a program-scoped UINT array; its dimension bounds the sequence length, and exceeding it would fault the controller, so index containment is a safety requirement rather than a nicety.

Data entry uses the thumbwheel gesture described earlier. A “maximum seen” latch captures the highest value dialled since the last store: rung 3 gates a `>` comparison with the debounce enable and moves the larger value into `PositionHold`. Dialling down through zero asserts `BCD_load_out`, and `FB_LoadArray` performs the edge-triggered store:

```
IF Inc_EN THEN
  IF (PositionHold_in > 0) AND (PositionHold_in <> 9) THEN
    PositionSequence_out[SequenceIndex] := PositionHold_in;
    SequenceIndex := SequenceIndex + 1;
    PositionMax_out := PositionMax_out + 1;
  END_IF;
  ...
ELSE
  IF (PositionHold_in = 9) THEN
    SequenceIndex := 1;
    PositionMax_out := 1;
  END_IF;
  ...
END_IF;
```

Value 9 is reserved as the sequence reset, which is why it is excluded from storage. Fig. 10 shows the capture and store rungs as built, and Fig. 11 the complete local variable set the finished Stage 5 program declares.

`FB_SequenceArray` then walks the array cyclically, using a modulo against the stored count and a `DetectedFlag` to guarantee exactly one advance per arrival:

```
IF (PositionSequence_in = PositionDetect_in)
  AND NOT DetectedFlag AND Inc_Seq_EN_in THEN
  SequenceIndexDint := MOD(SequenceIndexDint,
    SequenceMaxDint)+1;
  ...
  DetectedFlag := TRUE;
  Pickup_EN := ANY_TO_BOOL(1);
ELSIF (PositionSelect_in = PositionDetect_in)
  AND DetectedFlag AND Inc_Seq_EN_in THEN
  Place_EN := ANY_TO_BOOL(1);
  DetectedFlag := FALSE;
  ...
```

The block also emits `PickUp_EN` and `Place_EN` as one-scan event pulses (Fig. 9). These are not needed by the sequencer itself, but they are precisely the events Stage 6 requires, and exporting them here avoids duplicating the arrival detection later.

F. Stage 6: Process refinements

Stage 6 adds three independent capabilities on rungs 9–16.

Transfer delay. A real gripper needs dwell time at the transfer point. `PlaceDetectPulse` sets a latch `Wait_EN`, which enables a 500 ms TON; the timer’s Q output resets the latch. The intermediate signal `Partwait_Timing`, true only while the timer is running, is added as a normally-closed contact to both motor rungs, holding the wheel stationary for the delay.

Parts counter. A CTU counter clocked by `PickDetectPulse` accumulates processed parts in `PartsCount`. Because the count is intended for inventory rather than control, its reset is tied to the constant `FALSE` so the total survives stop and restart.

Cycle stop. A conventional stop halts the wheel wherever it is, potentially stranding a tool mid-transfer. The cycle-stop circuit inverts the seal-in of rung 1: it latches on `Stop` and unlatches on `Start`, and the latched `CycleStop` signal is placed in parallel with `RunState` on every rung that `RunState` gates. The process therefore continues after the stop request until two conditions coincide: at least one place event has occurred, and `SequenceIndex` has wrapped back

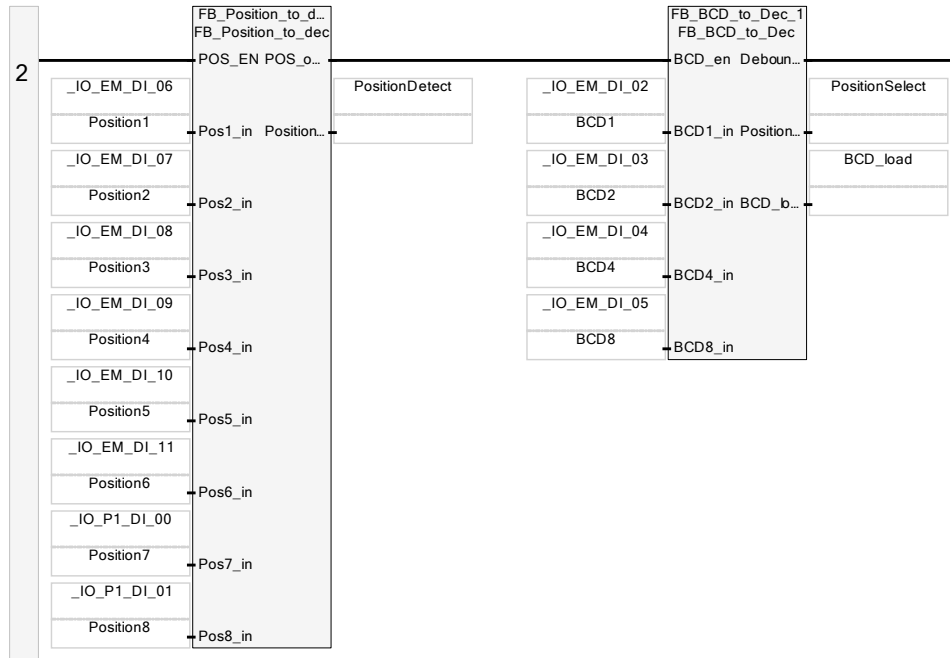


Fig. 5. Stage 3, rung 2 as built. FB_Position_to_dec reduces the eight one-hot position sensors to PositionDetect; FB_BCD_to_Dec reduces the four thumbwheel bits to PositionSelect and raises BCD_load at switch position 0.

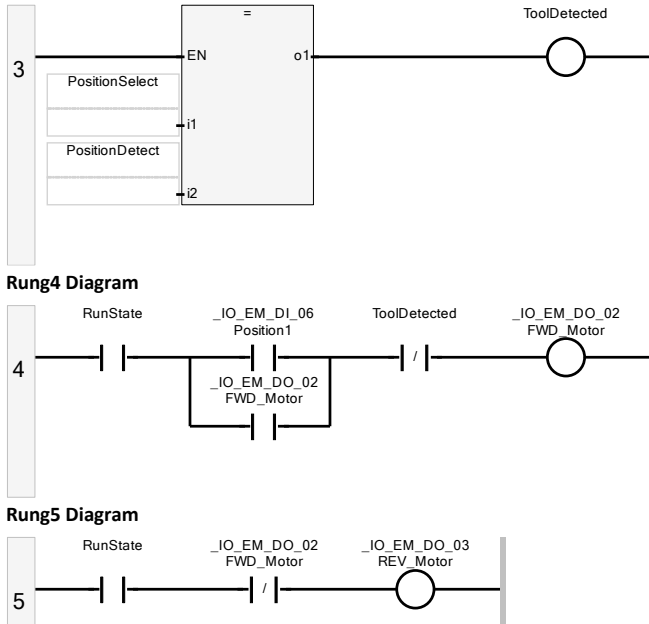


Fig. 6. Stage 3, rungs 3–5. The equality block replaces the hard-wired second sensor with a run-time comparison, and ToolDetected takes over the stopping role on rung 4.

to 1. At that point CycleStop_EN breaks the latch and the wheel halts at the base with the sequence complete.

The counter guard on the first condition exists to handle the degenerate case of Stop pressed immediately after Start, which would otherwise satisfy the index condition trivially and terminate before any work was done.

VI. SIMULATION TOOLING

Hardware trainers are shared resources, and iterating on an algorithm while occupying one is inefficient. Three self-contained HTML/JavaScript simulators were therefore developed, each requiring no build step, no network access at run time and no dependencies beyond a browser.

The Stage 4 simulator renders the wheel with base, target and antipode markers, animates the rotation, and steps through FB_DeterminRotation and FB_detectTool one decision at a time alongside a live rendering of rungs 1–5. It reimplements the antipode test directly, so agreement between it and the controller is a meaningful cross-check of the transcription:

```
function computeDir(base, target){
  var opp = ((base + 4) % 8) + 1, fwd;
  if (opp > base) fwd = (base < target) && (target < opp);
  else          fwd = !((base > target) && (target >=
    opp));
  return fwd ? 1 : -1;
}
```

The Stage 5 simulator adds the thumbwheel dial, a live view of PositionSequence[1..10], and separate tabs for the load path (rungs 3–5) and the run path (rungs 2, 6, 7, 8), which makes the two-phase nature of the operator workflow explicit.

The Stage 6 simulator renders the full sixteen-rung ladder in runtime-monitoring style, with power flow highlighted exactly as CCW displays it. Single-scan pulses are deliberately stretched to roughly 0.3 s so that they are visible; the timer, both counters and the cycle-stop latch animate against the turning wheel, and a scan log records each rung event with its scan number. It is the primary artefact for Stage 6, because,

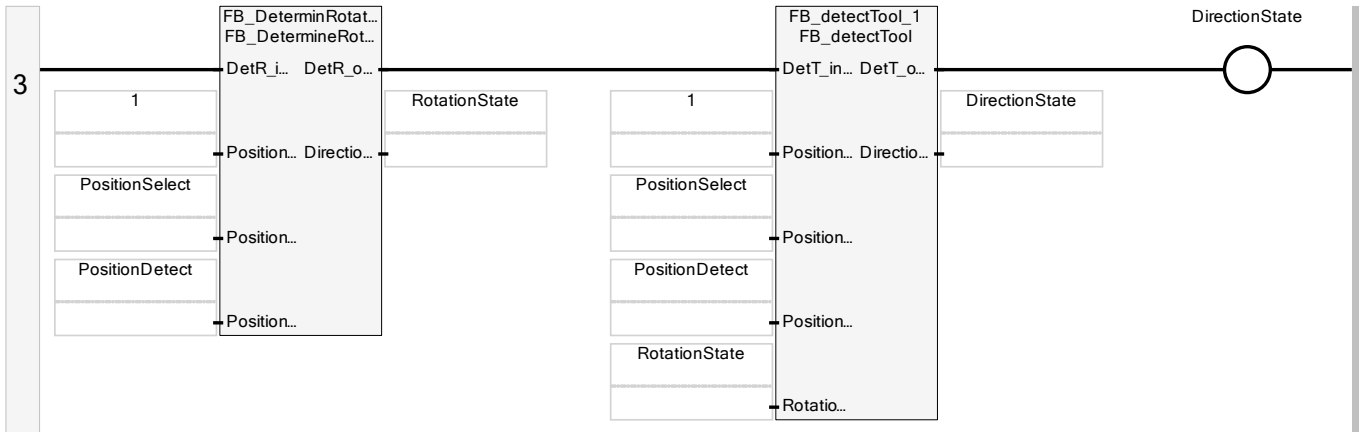


Fig. 7. Stage 4, rung 3 as built. The literal 1 on the first input of each block fixes station 1 as the base; PositionSelect carries the thumbwheel target and PositionDetect the live wheel position. Note the DirectionState coil terminating the rung (see Section VII, Finding 3).

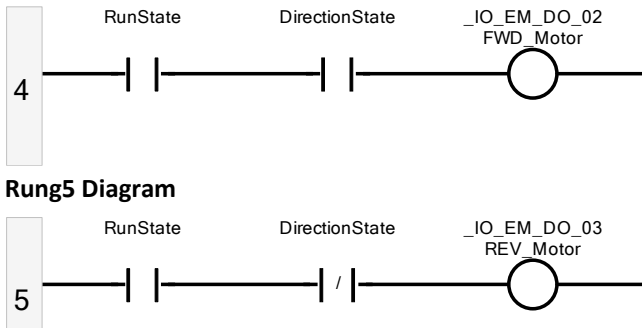


Fig. 8. Stage 4, rungs 4–5. With direction computed upstream, the motor rungs reduce to a complementary pair of two-contact rungs.

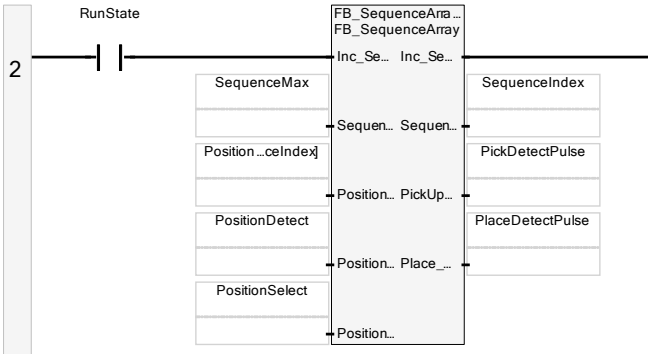


Fig. 9. Stage 5, rung 2 as built. PositionSequence[SequenceIndex] is an index-varying array reference, so the block reads whichever element the index currently selects and returns the advanced index along with the two event pulses.

as Section VII reports, the Stage 6 ladder was not committed to the controller project.

VII. VERIFICATION AND FINDINGS

A. Method

CCW's QuickLD compiler emits an intermediate Structured Text file (PROG1.txt) for every built ladder program. This

file is the ladder flattened into sequential assignments, in scan order, and it is a far more reliable object to review than the graphical diagram: branch topology, evaluation order and write ordering are all explicit. Each committed project was verified by reading this file against the intended behaviour, cross-referenced with the CCW Document Generator ladder printouts and the project variable lists.

The method is cheap, requires no hardware, and found three issues that inspection of the rendered ladder did not.

B. Finding 1: Stage 6 not present in the controller project

The Stage 6 CCW solution is byte-identical to Stage 5 in both Prog1.stf and the project variable list, and its generated ladder printout shows the same eight rungs. None of the Stage 6 additions exist in the controller project: the part-wait timer of rungs 9–12, the parts counter of rung 13, and the cycle-stop circuit of rungs 14–16 are all absent. Stage 6 is realised only in the browser simulator. The design is complete and validated in simulation; the transcription into CCW remains outstanding.

C. Finding 2: Off-by-one binding of rotation inputs

At Stage 5 the base position should migrate from a compile-time constant to the thumbwheel, so that the operator can change the transfer station while the sequence runs. The generated code reads:

```
FB_DeterminRotation_1(TRUE, 7, PositionSelect,
                      PositionSequence[SequenceIndex]);
```

Fig. 12 is the rung as built, and the literal 7 sitting on the first input of both blocks is visible directly. Comparing the actual against the intended binding, argument by argument:

	actual	intended
PositionSelect_in	7	PositionSelect
PositionSequence_in	PositionSelect	PositionSequence[SequenceIndex]
PositionDetectNumber_in	PositionSequence[...]	PositionDetect

The three arguments are shifted by one position. Two consequences follow. The base is frozen at station 7 rather

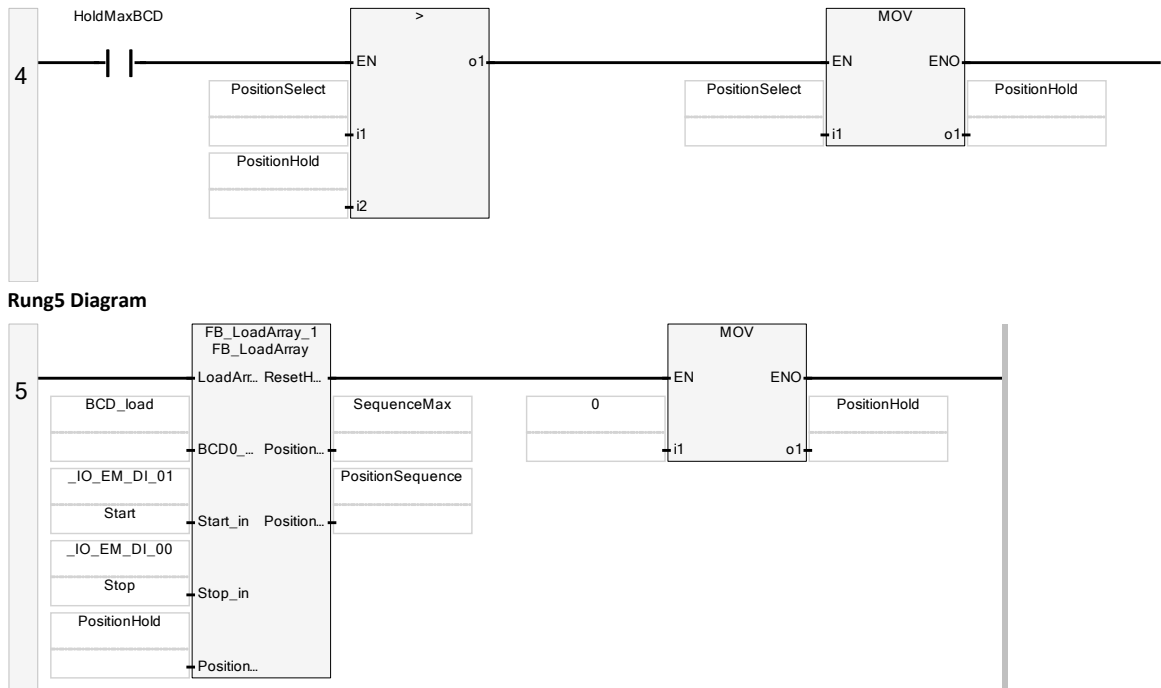


Fig. 10. Stage 5, rungs 4–5 as built. Rung 4 captures the largest debounced thumbwheel value into `PositionHold` through a `>` compare feeding a `MOV`; rung 5 stores that value into the array on the load pulse and then clears the hold with a second `MOV` of the constant 0.

Name	Alias	Data Type	Dimension	Initial Value	Project Value	Comment	Direction	Retained	String Size
RunState		BOOL					Var	False	
FB_Position_to_dec_1		FB_Position_to_dec		...	...		Var	False	
PositionDetect		UINT					Var	False	
FB_BCD_to_Dec_1		FB_BCD_to_Dec		...	...		Var	False	
BCD_load		BOOL					Var	False	
PositionSelect		UINT					Var	False	
DirectionState		BOOL					Var	False	
FB_DeterminRotation_1		FB_DeterminRotation		...	...		Var	False	
FB_detectTool_1		FB_detectTool		...	...		Var	False	
RotationState		UINT					Var	False	
HoldMaxBCD		BOOL					Var	False	
PositionHold		UINT					Var	False	
PositionSequence		UINT	[1..10]	...	...		Var	False	
SequenceMax		UINT					Var	False	
FB_LoadArray_1		FB_LoadArray		...	...		Var	False	
SequenceIndex		DINT					Var	False	
PlaceDetectPulse		BOOL					Var	False	
PickDetectPulse		BOOL					Var	False	
FB_SequenceArray_1		FB_SequenceArray		...	...		Var	False	

Fig. 11. Local variable declarations of the completed Stage 5 program. Six of the entries are function-block instances, `PositionSequence` carries the [1..10] dimension, and `SequenceIndex` is a `DINT` because the `MOD` operator requires one.

than tracking the thumbwheel. More seriously, the block's evaluation guard compares the current array element against the constant 7, instead of comparing the wheel sensor against the operator's base, so the direction decision is re-evaluated on the wrong event. `FB_detectTool_1` is bound identically and inherits the same defect. The correction is a three-field reassignment on one rung.

Comparing Fig. 12 against the Stage 4 equivalent, Fig. 7, shows how the error arose: the two rungs are structurally identical, and only the contents of the six input boxes changed

between stages. A constant in the first box looks correct because it looked correct at Stage 4.

D. Finding 3: Coil write overwrites function-block output

On the direction rung, `DirectionState` is assigned both as the output pin of `FB_detectTool` and as an output-energise coil terminating the rung. The compiler emits the two writes in that order:

```
DirectionState := FB_detectTool_1.DirectionState;
DirectionState := FB_detectTool_1.DetT_out_EN;
```

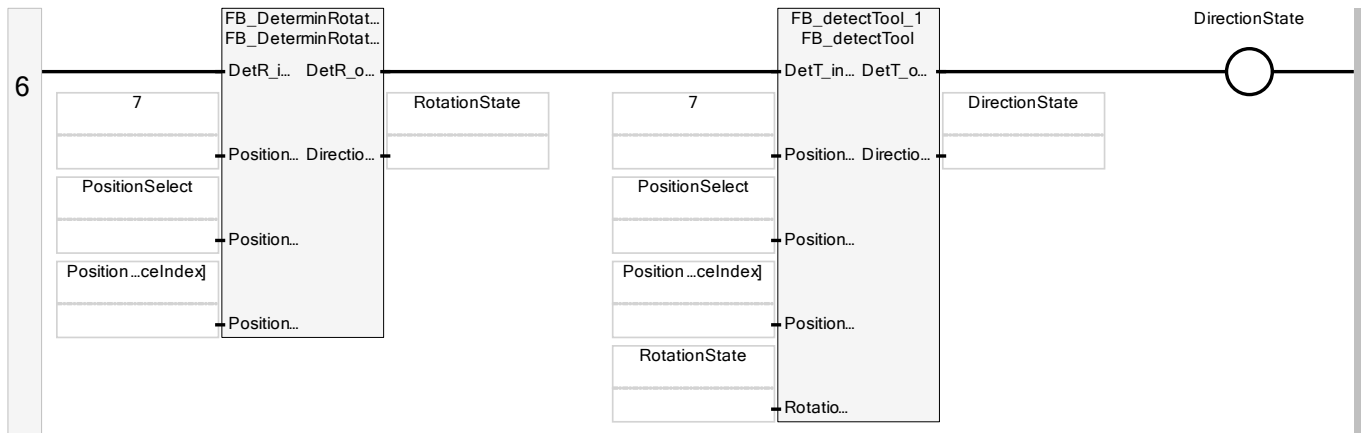


Fig. 12. Stage 5, rung 6 as built: the evidence for Finding 2. The first input of each block still holds the literal 7 carried over from the Stage 4 pattern, so `PositionSelect` and `PositionSequence[SequenceIndex]` each sit one slot lower than intended and `PositionDetect` is not connected to either block.

Because `DetT_out_EN` simply propagates rung power, the second write replaces the computed direction with the rung's power state. That write lands last, so it wins for the remainder of the scan. The motor rungs downstream consume the overwritten value.

This pattern reproduces the reference manual's own figures rather than originating in this implementation, and it is present from Stage 4 onward. The remedy is to remove the terminating coil and let the function-block output pin be the sole writer of `DirectionState`. It should be confirmed on hardware before the direction logic is relied upon, since a program in which the direction Boolean is pinned true would still appear to run; it would simply always rotate forward.

E. Discussion

All three findings share a characteristic: each is invisible in the graphical ladder and obvious in the generated text. Findings 2 and 3 in particular are aliasing errors, in which a correctly drawn diagram has bindings that do not mean what they appear to mean. This argues for treating the compiler's intermediate output as a first-class review artefact in PLC work, in the same way that a generated assembly listing is reviewed in safety-critical embedded development.

VIII. CONCLUSION

An automated pick-and-place tool magazine controller was developed on an Allen-Bradley Micro820 across six incremental stages, from a single-rung latch to a sixteen-rung program supported by six Structured Text function blocks. Partitioning the application so that ladder carries interlocking and sequencing while Structured Text carries arithmetic and decision logic kept the main program readable at every stage while permitting genuinely algorithmic behaviour, including a modulo-based shortest-arc rotation decision whose sole ambiguity was characterised and bounded: one equidistant target per base, an unavoidable consequence of the wheel's even order.

Three browser-based simulators were built to exercise the algorithm and the ladder without occupying the hardware trainer, and proved valuable both for design iteration and, in the case of Stage 6, as the working record of a design not yet transcribed to the controller.

Verification by reading the ladder compiler's intermediate Structured Text output proved markedly more effective than inspection of the graphical diagram, yielding three findings that are reported in full above. Completing the Stage 6 transcription into CCW, correcting the function-block argument binding at Stage 5, and eliminating the redundant direction coil are the outstanding work items. All three are localised changes, and the simulators provide a ready means of validating them before the trainer is next available.

ACKNOWLEDGMENT

The lab manual is the work of Dr. Sangram Redkar, Instructor of Manufacturing Systems and Networks, Ira A. Fulton Schools of Engineering, Arizona State University, AZ. *Lab Documentation v1.1* is distributed with the trainer by plccable.com. The six Structured Text function blocks are transcribed from Appendix A of that manual.

REFERENCES

- [1] S. Redkar, *Industrial Concepts CNC Tool Magazine Control Unit: Lab Documentation v1.1, with training in Connected Components Workbench*. Tempe, AZ: Ira A. Fulton Schools of Engineering, Arizona State University; distributed with the trainer by plccable.com.
- [2] International Electrotechnical Commission, *IEC 61131-3: Programmable Controllers, Part 3: Programming Languages*, 3rd ed. Geneva, Switzerland: IEC, 2013.
- [3] Rockwell Automation, *Micro830, Micro850 and Micro870 Programmable Controllers User Manual*, Publication 2080-UM002. Milwaukee, WI: Rockwell Automation.
- [4] Rockwell Automation, *Connected Components Workbench Software User Manual*, Publication 9328-UM001. Milwaukee, WI: Rockwell Automation.
- [5] Rockwell Automation, *Micro800 Plug-in Modules Installation Instructions*, Publication 2080-IN001. Milwaukee, WI: Rockwell Automation.